



홍수빈

인프라 엔지니어 · SRE / DevOps

노트북 한 대에 k3s 클러스터를 세워 예매 대기열 서비스를 인터넷에 공개하고, 10,000명 부하에서 5xx 0건을 유지하며 정원·자원 스펙을 실측했습니다.

Portfolio	subinhong.dev
GitHub	github.com/sss654654
Email	subinhong0109@gmail.com
Phone	010-5481-6740

Skills

Kubernetes	k3s · etcd · Helm · Calico · MetalLB · Traefik · NetworkPolicy
CI/CD · GitOps	GitLab CI / Runner · ArgoCD · argocd-image-updater · Sealed Secrets · Trivy · gitleaks
Observability · Load	Grafana · Mimir · Loki · Tempo · Alloy · Pyroscope · OpenTelemetry · PromQL / LogQL / TraceQL · k6
IaC · Cloud	Terraform · AWS (VPC · VPC Endpoint · Client VPN · Kinesis · ECR · IRSA)
Network · Security	OPNsense · WireGuard · Cloudflare · cert-manager · Let's Encrypt
Virtualization · OS	Proxmox VE · KVM / QEMU · LVM · Ubuntu Server
Backend · Data	Go · Java (Spring Boot) · Redis · Kafka · MySQL

Experience

semiai · 인프라팀	2026.03 – 2026.06
온프레미스 k3s 클러스터의 관측 계층 구성 — 기구축된 k3s-LGTM 스택 위에서 계측·수집·대시보드 담당	
k3s · Grafana · Mimir · Loki · Tempo · Pyroscope · Alloy · OpenTelemetry · Go · ArgoCD · MinIO	
– [수집 통합] 신호마다 따로 있던 수집기 넷(Prometheus · Promtail · OTEL Collector · Pyroscope agent)을 Alloy 하나로 통합, 저장은 MinIO(S3)로 영속화 – [클러스터 대시보드] control-plane 생존 → 비정상 Pod → 자원 한계 임박 순으로 좁혀 가는 데일리 대시보드 설계 — 비정상 Pod 식별은 실패 유형 6종 OR 조건 대신 kube_pod_status_ready 단일 메트릭으로 판정 – [장애 미표시] master 2대 정지 후에도 대시보드가 Ready로 남는 현상을 확인 — kube-state-metrics 경유 대신 kubelet · etcd 직접 scrape로 전환 – [앱 계측] Go 백엔드에 metric · trace · log · profile 계측을 추가하고 trace_id로 잇는 진단 동선 구성 — trace_id를 메트릭 라벨에 넣으면 시계열이 폭발해, metric→trace는 path 라벨로 TraceQL 검색을 거는 dataLink로 연결 – [알림] 개발자만 사용하는 dev 환경이라 표본이 적어 비율·분위수 기준이 의미 없고, 5xx 절대 건수를 알림 기준으로 두어 해당 요청의 trace · log로 이어지도록 연결	
cloudwave 클라우드 부트캠프 · CJ 올리브네트웍스	2025.06 – 2025.09
CGV 예매 대기열 시스템 — 5인 팀 · 개발계 네트워크 계층(Terraform)과 대기열 백엔드(Spring Boot) 담당	
github.com/sss654654/dev_terraform · dev_backend	
Terraform · AWS (VPC · VPC Endpoint · Client VPN · EKS · Kinesis · ECR) · Spring Boot · Redis · WebSocket	
– [네트워크] VPC · 서브넷 6개 · 라우트 테이블 4개 · 보안그룹 5개 · VPC 엔드포인트를 Terraform으로 구성 — ECR 트래픽이 NAT를 거치지 않도록 ecr.api · ecr.dkr 엔드포인트를 서브넷별로 배치 – [대기열] Redis Sorted Set 둘(waiting · active)로 대기과 입장을 분리하고 승격 이벤트는 Kinesis로 전달 — 정원은 Pod 수 기반 동적 계산 – [Kinesis 스로틀] ProvisionedThroughputExceededException 반복 — 원인은 샤드 부족만이 아니라 Consumer가 0번 샤드만 읽는 구조 → Pod별 라운드로빈 멀티샤드 소비로 재작성하고 샤드 1 → 2 – [IRSA] Pod의 Kinesis 접근이 AccessDenied — 서비스 계정 · 신뢰 관계는 정상, 원인은 spring-cloud-aws-starter 의존성 누락 – [검증] 입장 요청을 스크립트로 100 → 1,000 → 10,000명 세 단계로 투입, HPA 확장과 대기열 동작 확인	

Certifications

AWS Developer Associate 2025.09 · AWS Solutions Architect Associate 2024.11 · 정보처리기사 2024.06
리눅스마스터 2급 2023.12 · SQLD 2022.12

Education

단국대학교 소프트웨어학과
2021.03 – 2026.02 · 학점 3.76 / 4.5

Military Service

공군 병장 만기전역
2023.08 – 2025.05

Projects

온프레미스 k3s 홈랩 — 대기열 서비스 구축 · 공개 · 부하 실측

2026.07 – 2026.09

노트북 1대(RAM 32GB)에 Proxmox → VM 3대 → k3s — 물리 계층부터 인터넷 공개까지 단독 구축 · ticket.subinhong.dev 운영 중(08–23시 KST)

subinhong.dev/homelab · github.com/sss654654/cgv-infra · [cgv-onprem](https://cgv-onprem.com)

Proxmox · k3s · Calico · MetalLB · Traefik · GitLab CI · ArgoCD · Mimir · Loki · Tempo · OPNsense · k6

- **[클러스터]** VM 3대를 control-plane 겸 etcd 멤버로 k3s 구성 — 파드 네트워크 · 로드밸런서 · 스토리지를 k3s 기본 대신 Calico · MetalLB · 정적 PV로 교체, Traefik은 번들 대신 직접 배포
- **[CI/CD]** GitLab CI 5단 → ArgoCD GitOps 배포 — 파이프라인 한 판 **10분 15초 → 1분 54초**, 커밋에서 클러스터 반영까지 **3분 → 3초**
- **[관측]** Alloy 하나로 수집해 Mimir · Loki · Tempo에 저장, 클러스터 · 호스트 대시보드 2장과 Discord 알림 — 호스트 지표를 붙여 유휴 CPU **92°C**의 원인을 찾아 **66°C**로 낮춤
- **[서비스]** 대기열 queue(Go)와 예매 booking(Spring)을 분리해 Kafka로 연결, 코드에 metric · trace · log 계측 추가
- **[격리 · 공개]** OPNsense 방화벽 VM과 가상 브리지로 노드를 격리망에 배치 — 관리 UI는 WireGuard 터널 안에서만, 서비스는 Cloudflare 엣지 경유 요청만 통과, 인터넷에 열린 포트는 2개로 제한
- **[부하 실측]** SLO(5xx 0 · 입장 p99 1초 · 폴링 p99 3초)를 먼저 정하고 k6로 실제 여정 그대로 부하 투입 — 인원 1,000 → 10,000 · 정원 2 → 1,000 계단으로 **10,000명 · 735,273 요청 · 5xx 0건** (30,000명은 노드 RAM 8GB × 3 상한에 막힘)
- **[트러블슈팅]** JIT 컴파일 전 부하 도착으로 booking p99 4.80초 → 측정 절차에 워업 추가 후 0.94초 · Redis 커넥션 풀 10 → 50으로 풀 대기 누적 10,952초 → 785초

Research

LevelDB Cache Mechanism Analysis · 한국대 System Software Lab

2022.07 – 2022.12

Google LevelDB 캐시 메커니즘 C++ 소스 분석 · **KSC 2022 학부생 논문 1저자** · 포스터 발표

github.com/sss654654/leveldb-cache-analysis

C++ · LevelDB · LSM-tree · LRU Cache · db_bench

- 인덱스 캐시(개수 기준)와 블록 캐시(바이트 기준)가 ShardedLRUCache 하나를 공유하는 구조를 소스 레벨에서 분석 — 16-shard 독립 락 · 해시테이블 + 이중 연결 리스트 공유 노드
- db_bench로 캐시 개수 · 크기를 독립 변수로 두고 워크로드 셋(readrandom · seekrandom · readhot) 측정 — 인덱스 캐시는 SSTable 수에서 포화, 블록 캐시는 워킹셋에 비례해 개선(readrandom **190 → 27μs**)